



Automated Peptide Filtration for Immunoinformatics-Based Vaccine Prediction Studies: From Full-Length to Core

Post-Publication Automation of Peptide Filtration

Part 1: Introduction to IEDB:

Immune Epitope Database (IEDB): The IEDB Database and Analysis Resource is a publicly available resource for studying experimentally characterized immune epitopes and performing computational epitope prediction. It provides a comprehensive database of immune epitope data together with analysis and prediction tools for **B-cell** and **T-cell** epitopes. IEDB supports several stages of immunoinformatics-based vaccine design, including **B-cell epitope prediction**, **T-cell epitope prediction**, **MHC binding analysis**, **population coverage**, **epitope conservation**, and **epitope comparison**.

B-Cell Epitope Prediction: B-cell epitopes are regions of an antigen that can be recognized by **B-cell receptors or antibodies**. IEDB provides both **sequence-based and structure-based approaches** for predicting B-cell epitopes. For **linear B-cell epitope prediction**, IEDB analyzes antigen sequences using methods based on amino acid properties and sequence characteristics. Current IEDB tools include **BepiPred 3.0** and several amino acid scale-based approaches. In our publication, we used **BepiPred 2.0, the last version of the tool**.

Moreover, for **structure-based prediction**, IEDB provides methods such as **DiscoTope** and **ElliPro**, which use structural characteristics such as solvent accessibility, residue contacts, and structural protrusion to identify potential antibody-accessible epitopes. For more details: <https://discuss.iedb.org/t/b-cell-epitope-prediction-tools-description/146>

T-Cell Epitope Prediction: T-cell epitope prediction focuses mainly on identifying peptides that can be presented by **MHC molecules** and potentially recognized by T cells. IEDB provides separate prediction workflows for **MHC class I** and **MHC class II**. Importantly, predicted MHC binding is a key component of T-cell epitope prediction, but **MHC binding alone is not sufficient to establish T-cell recognition**.

MHC Class I: CD8⁺ T-cell responses: MHC class I molecules primarily present intracellularly derived peptides to **CD8⁺ cytotoxic T cells**. IEDB provides MHC-I binding prediction tools that evaluate the ability of peptide sequences to bind specific MHC-I alleles. The current IEDB recommended approach for MHC-I peptide binding uses **NetMHCpan 4.1 BA**, while **NetMHCpan 4.1 EL** is used for epitope prediction because elution-based predictions incorporate information related to naturally presented peptides.

MHC Class II: CD4⁺ T-cell responses: MHC class II molecules present peptides primarily to **CD4⁺ helper T cells**. IEDB provides MHC-II binding prediction tools for HLA-DR, HLA-DP, and HLA-DQ molecules, as well as mouse MHC-II alleles. MHC-II binding prediction differs from MHC-I because the MHC-II binding groove is open-ended, allowing peptides of variable length to bind around a shorter **core binding region**. IEDB's tools therefore evaluate peptide–MHC-II



interactions and identify peptides with favorable predicted binding. The current IEDB-recommended MHC-II approach uses **NetMHCIIpan 4.1 EL for epitope prediction** and **NetMHCIIpan 4.1 BA for binding prediction**. In our study, we used the ANN.40 prediction method.

For more details:

- 1- [https://tools.iedb.org/mhci/help/#:~:text=Currently%20for%20peptide%3A%20MHC%2DI%20binding%20prediction%2C%20NetMHCpan,predictors%20for%20binding%20\(BA\)%20and%20elution%20\(EL\).](https://tools.iedb.org/mhci/help/#:~:text=Currently%20for%20peptide%3A%20MHC%2DI%20binding%20prediction%2C%20NetMHCpan,predictors%20for%20binding%20(BA)%20and%20elution%20(EL).)
- 2- https://nextgen-tools.iedb.org/docs/tools/tcell_ii/index.html

Part 2: Getting Started: Prepare your row peptide files

B-cell row file Preparation:

- 1- Open the IEDB site <https://www.iedb.org/>

Summary Metrics

Peptidic Epitopes	1,907,728
Non-Peptidic Epitopes	3,412
T Cell Assays	577,219
B Cell Assays	1,689,641
MHC Ligand Assays	5,774,266
Epitope Source Organisms	4,816
Restricting MHC Alleles	1,021
References	27,046

[Submit Help/Feedback](#) | [Support](#) | [Privacy Policy](#)

Supported by a contract from the National Institute of Allergy and Infectious Diseases, a component of the National Institutes of Health in the Department of Health and Human Services.



Last Updated: August 09, 2026

Click on the Antigen Sequence Properties in the **B Cell Epitope Prediction** section in the last version.

For more details, look at this video that explains the B-Cell test:

<https://www.youtube.com/watch?v=gsz230yUOOw>



2- Enter your sequence into the IEDB analyzer

<https://tools.iedb.org/bcell/>

IEDB Analysis Resource

Home
Help
Example
Reference
Download
Contact

Antibody Epitope Prediction

Specify Input

Enter a Swiss-Prot ID (example: P02185)

Or enter a protein sequence in plain format (50000 residues maximum, 250 residues for Bepipred 2.0):

MPAMPSSGPGDTSSSAAREEDRKDGEEQEPRGKEERQEPSTTARKVGRPGRKRPVSGDTPKDPA
 VISKSPSMAQDSGASELLPNGDLEKRSEPQPEEGSPAGGQKGGAPAEGEAAETLPEASRAVENGCCTPK
 EGRGAPAEAGKEQKETNIESMKMEGSRGRLRGGLSWESSLRQRPMPRLTFQAGDPYYISKRRDEWLARW
 KREAEEKAKVIAGMNAVEENQGPGESQKVEEASPAVQQPTDPASPTVATTPEPVGSDAGDKNATKAGDD
 EPEYEDGRGFGIGELVWGKLRGFSWPGRIVSWMTGRSRAAEGTRWVMWFGDGKFSVVCVEKLMPLSSF
 CSAFHQATYNKQPMYRKAIYEVQLQVASSRAGKLPVCHDSDESDTAKAVEVQNKPMIEWALGGFQPSGPK
 GLEPPEEEKNPYKEVYTDMMWEPEAAAYAPPPAKKPRKSTAEKPKVKEIIDERTRELVEVVRQKCRNI
 EDICISCGSLNVTLEHPLFVGGMQCNCKNCFLECAVQYDDDGYSYCTICCGGREVLGCGNNCCRCFCV
 ECVDLLVGPAAQAIAIKEDPWNCYMGHKGTYGLRRREDWPSRLQMF FANNHDQEFDPKVPYPPVPAEK
 RKPIRVLSLFDGIATGLLVKDLGIQVDRIASEVCEDSITVGMVRHQGKIMYVGDVRSVTQKHIQEWGP

Choose a method:

☒ [Bepipred Linear Epitope Prediction 2.0](#)

☐ [Bepipred Linear Epitope Prediction](#)

☐ [Chou & Fasman Beta-Turn Prediction](#)

☐ [Emini Surface Accessibility Prediction](#)

☐ [Karplus & Schulz Flexibility Prediction](#)

☐ [Kolaskar & Tongaonkar Antigenicity](#)

☐ [Parker Hydrophilicity Prediction](#)

Enter your sequence in the box and click on the Bepipred Linear Epitope Prediction 2.0, then submit.

Submit

Reset

*Your prediction has been submitted. This page will automatically update when results are ready. You may bookmark this page and return later.



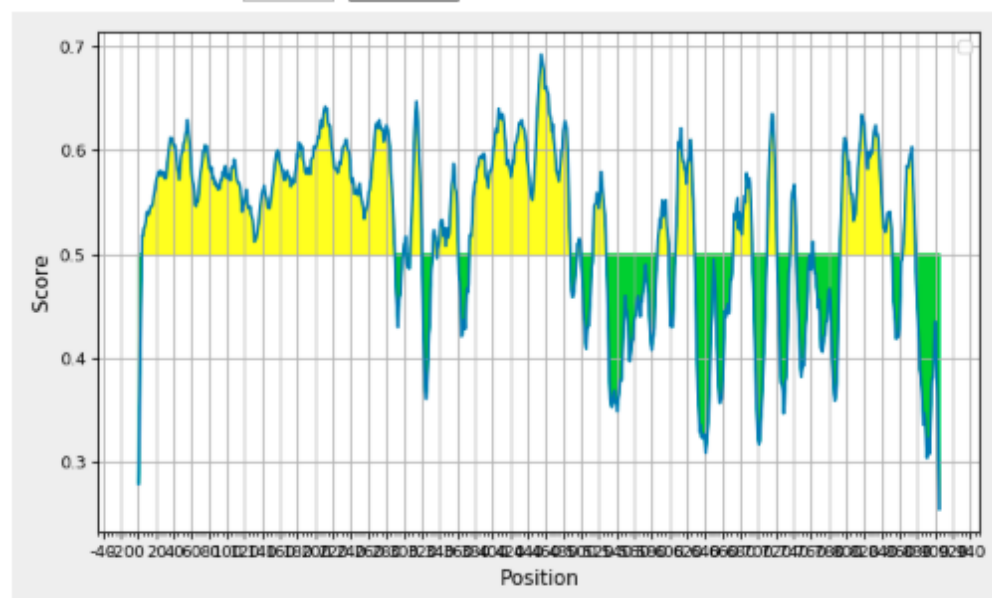


Bepipred Linear Epitope Prediction 2.0 Results

Input Sequences

```

1 MPAMPSSGPG DTSSAAERE EDKRDGEEQE EPRGKEERQE PSTTARKVGR PGRKRKHPPV
61 ESGDTPKDPV VISKSPSMAQ DSGASELLPN GDLEKRSEPO PEEGSPAGGQ KGGAPAELEG
121 AAEITLPEASA VENGCTPKE GRGAPAEAGK EQKETNIESM KMEGSRGRLR GGLGWESSLR
181 QRPMPRLTFQ AGDPYYSKR KRDEWLARK REAEKKAKVI AGMNAVEENQ GPGESQKVEE
241 ASPPAVQOPT DPASPVATT EPVGSADGDK NATKAGDEPE EYEDGRGFGI GELVWGKLRG
301 FSWMPGRIVS WMTGRSRAA EGRWVMVFG DGKFSVVCVE KLMPLSFCS AFHQATYNKQ
361 PMYKAIYEV LQVASSRAGK LPVCHDSDES DTAKAVEVQN KPMIEWALGG FQSPGPKGLE
421 PPEEEKNPYK EYTDWVVEP EAAAYAPPP AKKPKSTAE KPKVKEIIDE RTRERLVYEV
481 RQKCRNIEDI CISCGLNVT LEHPLFVGM CQNCNCFLE CAYQYDDGY QSYCTICGG
541 REVLGCGNN CCRCFCVECV DLLVGPAAQ AAIKEDPWC YMGHGKTYG LLRRREDWPS
601 RLQMFANHH DQEFDPKVV PPVPAEKRP IRVLSLFDGA TGLLVKDLG IQVDRIASE
661 VCESITVGM VRHQKIMYV GDRSVTQKH IQEWGPFDLV IGGSPCNDLS IVNPARKGLY
721 EGTGRLLFEF YRLLHDARPK EGDRPFVNL FENVVAMGV DKRDISRFLE SNPVMIDAKE
781 VSARARYFMG NLPGMNRPLA STVNDKLELQ ECLEHGRIAK FSKVRTITTR SNSIKQGDQ
841 HFPVFMNEKE DILWCTEMER VFGFPVHYTD VSNMHLARQ RLLGRSWSVP VIRHLFAPLK
901 EYFACV
  
```

Center position: 4 Threshold: 0.500 

Average: 0.527 Minimum: 0.255 Maximum: 0.692

Predicted peptides:

No.	Start	End	Peptide	Length
1	5	300	ESGDTPKDPV VISKSPSMAQ DSGASELLPN GDLEKRSEPO PEEGSPAGGQ KGGAPAELEG AAEITLPEASA VENGCTPKE GRGAPAEAGK EQKETNIESM KMEGSRGRLR GGLGWESSLR QRPMPRLTFQ AGDPYYSKR KRDEWLARK REAEKKAKVI AGMNAVEENQ GPGESQKVEE ASPPAVQOPT DPASPVATT EPVGSADGDK NATKAGDEPE EYEDGRGFGI GELVWGKLRG FSWMPGRIVS WMTGRSRAA EGRWVMVFG DGKFSVVCVE KLMPLSFCS AFHQATYNKQ PMYKAIYEV LQVASSRAGK LPVCHDSDES DTAKAVEVQN KPMIEWALGG FQSPGPKGLE PPEEEKNPYK EYTDWVVEP EAAAYAPPP AKKPKSTAE KPKVKEIIDE RTRERLVYEV RQKCRNIEDI CISCGLNVT LEHPLFVGM CQNCNCFLE CAYQYDDGY QSYCTICGG REVLGCGNN CCRCFCVECV DLLVGPAAQ AAIKEDPWC YMGHGKTYG LLRRREDWPS RLQMFANHH DQEFDPKVV PPVPAEKRP IRVLSLFDGA TGLLVKDLG IQVDRIASE VCESITVGM VRHQKIMYV GDRSVTQKH IQEWGPFDLV IGGSPCNDLS IVNPARKGLY EGTGRLLFEF YRLLHDARPK EGDRPFVNL FENVVAMGV DKRDISRFLE SNPVMIDAKE VSARARYFMG NLPGMNRPLA STVNDKLELQ ECLEHGRIAK FSKVRTITTR SNSIKQGDQ HFPVFMNEKE DILWCTEMER VFGFPVHYTD VSNMHLARQ RLLGRSWSVP VIRHLFAPLK EYFACV	296
2	301	303		3
3	308	321		14

3- Now, download the required file: start with the predicted peptide table as raw data, then save your file in CSV format for automated filtering. Just include the start, end, and peptide, and adjust the length to a specified number.

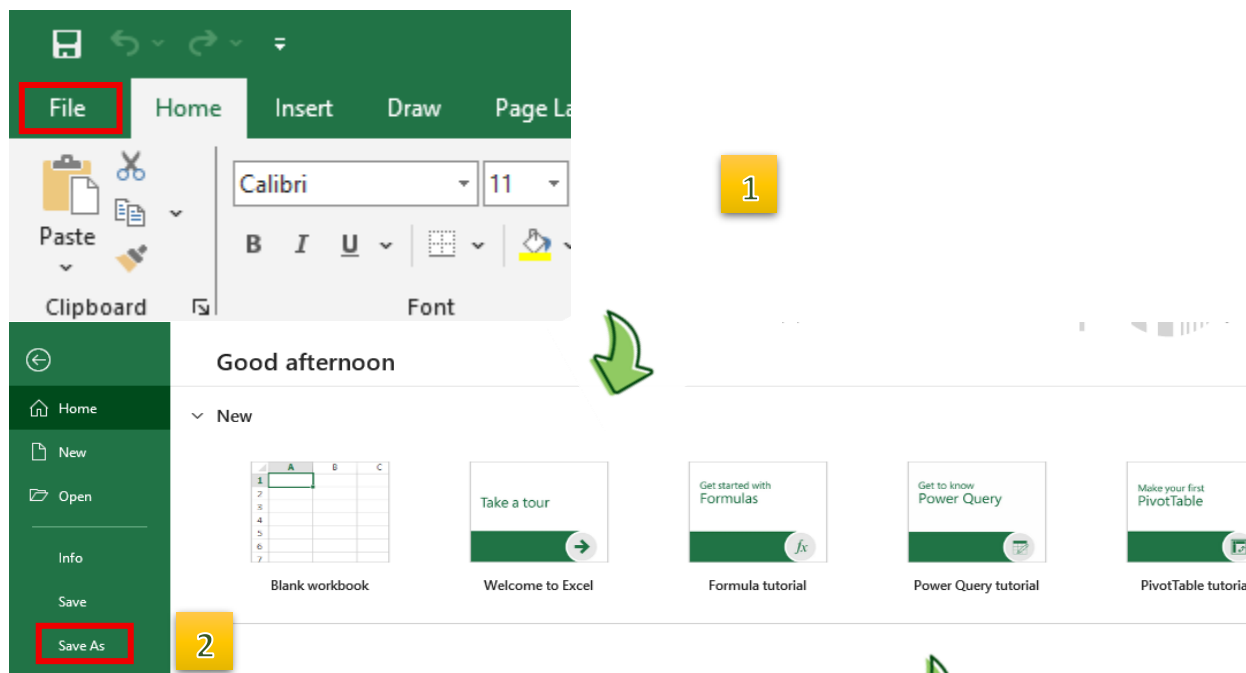
GET GENUINE OFFICE Your license isn't genuine, and you may be a victim					
	A	B	C	D	E
1	Start	End	Peptide	number	
2	1	158	MPAMPSS	158	
3	163	170	MEGSRGR	8	
4	179	183	SLRQR	5	
5	190	198	FQAGDPY	9	
6	200	203	KRRR	4	
7	212	217	REAEKK	6	
8	226	290	AVEENQG	65	
9	318	324	RSRAAEG	7	
10	358	364	TYNKQPM	7	



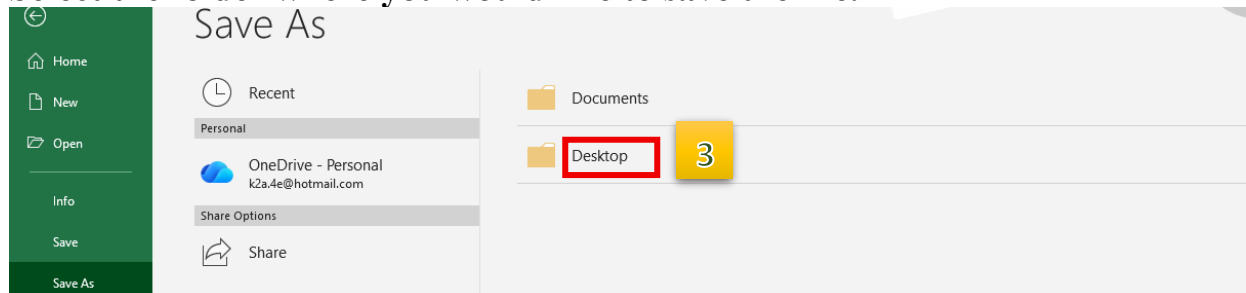
Your peptide is now prepared for the automated Conical Peptide Generator: truncate peptides to a fixed length of 4.



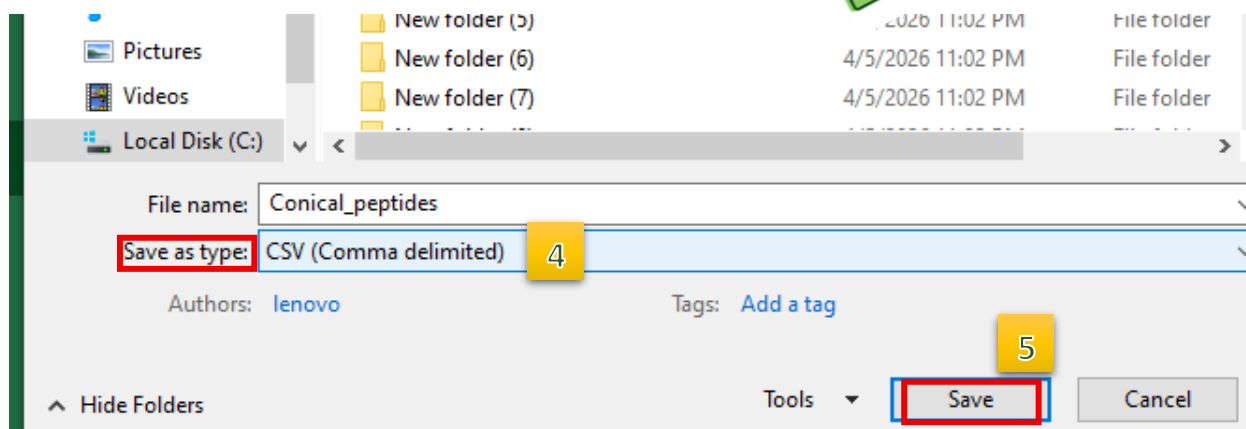
4- Here's an explanation of how to save your raw file in CSV format from Excel. **Click on.**



Select the folder where you would like to save the file.



From Save type, select CSV (Comma delimited).





2- T cell predictions row file:

- 1- Open the IEDB site: <https://www.iedb.org/>
- 2- Navigate to the “MHC Class I Tools Suite” (Immunogenicity, Processing, Binding/Elution). This module enables the extraction of all MHC Class I predicted peptides and their corresponding HLA allele assignments.
- 3- Do the same steps for the MHC Class II

- 4- Choose all 27 alleles, then select the appropriate prediction methods for your test, such as ANN 4.0. **peptide length 11**



Peptide Length(s) 8 9 10 11 12 13 14 ☐ as-is

MHC Allele(s) Ex: HLA-A*02:01

Prediction Model(s)

MHC-I Binding/Elution

NetMHCpan 4.1 EL (recommended epitope predictor-2023.09)
NetMHCpan 4.1 BA (recommended binding predictor-2023.09)
Consensus
ANN 4.0
SMPMBEC
SMM
CombLib_Sidney2008
MHCFlurry 2.0
MHC-NP
NetMHCpan 4.1 EL (recommended epitope predictor-2023.09)



Prediction Model(s)

MHC-I Binding/Elution

Prediction Method
ANN 4.0

Add Another Prediction

Run

Results will load once this step is run.



Peptide Table Sequence Table

Download Reset Table Display Columns Save Table State

Show 50 rows

Previous 1 2 3 4 5 ... 76 Next

seq #	peptide	start	end	peptide length	allele	media bindin percen
1	YLQPRTEFL	269	277	9	HLA-A*02:01	
1	FQFCNDPFL	133	141	9	HLA-A*02:01	
1	FIAGLIAIV	1220	1228	9	HLA-A*02:01	
1	FVSNQTHWFV	1095	1104	10	HLA-A*02:01	
1	SIIAYTMSL	691	699	9	HLA-A*02:01	

Your prediction has been submitted. Download it.



5- Next, extract the peptides and alleles columns and save them in a new CSV file format for arranging using the automated method.

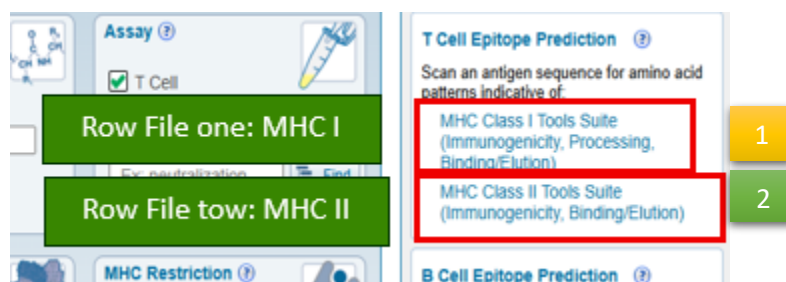
seq #	peptide	start	end	peptide le	allele	peptide ir	median bi	ann IC50	ann percentile										
1	YLQPRFTLL	269	277	9	HLA-A*02:01	269	0.03	5.36	0.03										
1	FQFCNDPFL	133	141	9	HLA-A*02:01	133	0.07	9.18	0.07										
1	FIAGLIAIV	1220	1228	9	HLA-A*02:01	1220	0.09	10.29	0.09										
1	FVSNQTHWV	1095	1104	10	HLA-A*02:01	2360	0.11	13.1	0.11										
1	SHIAYTMSL	691	699	9	HLA-A*02:01	691	0.12	13.54	0.12										
1	KLNDLCFTNV	386	395	10	HLA-A*02:01	1651	0.14	15.27	0.14										
1	RLQSLQTYV	1000	1008	9	HLA-A*02:01	1000	0.16	16.66	0.16										
1	YHLMSPQSA	1047	1056	10	HLA-A*02:01	2312	0.17	17.89	0.17										
1	YIKWPWYWL	1209	1218	10	HLA-A*02:01	2474	0.18	18.81	0.18										
1	YIWLGIAGL	1215	1224	10	HLA-A*02:01	2480	0.19	20.48	0.19										
1	FELHAPATV	515	524	10	HLA-A*02:01	1780	0.19	20.98	0.19										
1	FTISVTTEI	718	726	9	HLA-A*02:01	718	0.22	25.37	0.22										
1	LLFNKVTIA	821	829	9	HLA-A*02:01	821	0.22	25.41	0.22										
1	HLMSFPQSA	1048	1056	9	HLA-A*02:01	1048	0.23	26.36	0.23										
1	FVFLVLLPLV	2	11	10	HLA-A*02:01	1267	0.29	32.64	0.29										
1	VLNDILSRL	976	984	9	HLA-A*02:01	976	0.3	33.57	0.3										
1	FIEDLLFNKV	817	826	10	HLA-A*02:01	2082	0.32	36.03	0.32										
1	GYLQPRFTLL	268	277	10	HLA-A*02:01	1533	0.32	36.12	0.32										
1	KIADYNYKL	417	425	9	HLA-A*02:01	417	0.32	36.12	0.32										

6- The input files for the scripts is ready now

Example Input:

peptide	allele
HDSDSDTAKA	HLA-A*02:01
HDSDSDTAKA	HLA-B*07:02
HDSDSDTAKA	HLA-A*02:01

7- Note that you will repeat the same steps to get the MHCII row file. You will have two raw files: MHCI and MCHII.



Preparing the Computational Environment:

Following the IEDB prediction stage, three output files are generated: a B-cell epitope prediction file, an MHC class I prediction file, and an MHC class II prediction file. These files serve as the input for the next stage of the workflow. To avoid repetitive manual filtering, a **Linux** environment is prepared, and **Python** is installed with the required libraries. The Python scripts are then used to automatically process and filter the prediction outputs, making the workflow faster, reproducible, and less prone to manual errors.



Part 3: Getting Started: Your Linux Environment

Objective: Understand and explore the Linux terminal.

1- Open the Terminal and check where you are.

Commands:

```
# Print Working Directory
$ pwd
```

Explanation:

pwd (Print Working Directory) shows your current folder location.

2- Explore available files using ls.

Commands:

```
# Explore available files
$ ls
```

```
# Explore available files
$ ls
$ ls -l
$ ls -a
```

Explanation:

- **ls** List all the contents of the current directory (by default).
- **ls-l** Long listing (shows permissions, dates, sizes).
- **ls-a** Include hidden files.

Part 4: Creating directories, Navigating through the File System

Objective: Create and manage directory/files. Learn file navigation using absolute and relative paths. **Commands:**

```
# Make a directory
$ mkdir Immunoinformatics
# change directory
$ cd Immunoinformatics
# Make a directory
$ mkdir Data Results Scripts AI
# Remove empty directory
$ rmdir Results
# Remove directory and contents
$ rm -r AI
```



Explanation:

- **mkdir**: Make a directory; create a new directory or file.
- **rmdir** removes empty folders.
- **rm -r** removes a folder with its contents.

Part 5: Starting: Python script & set up the environment:

Objective: Understand using Python script.

Why choose a Python script instead of another scripting language?

Python is often the preferred language for **bioinformatics** and data science for several important reasons:

1- Powerful Libraries

- Python has **bioinformatics-specific libraries** like:

 **Biopython** for sequence analysis

 **pandas** for data handling (e.g., CSV files)

 **numpy** for numerical operations.

- These tools help **solve biological problems quickly and efficiently**.

2- Widely Used in Research

 Most modern bioinformatics pipelines and tools (e.g., Galaxy, Jupyter, scikit-bio) are written in or support Python.

 Python is used by many institutions like EMBL-EBI, NCBI, and universities worldwide.

3- Cross-Platform and Open Source

 Python runs on Linux, Mac, and Windows.

 Free, use it anywhere without a license.



1- Install pip for Python 3 on Ubuntu (or other Debian-based systems):

```
$ sudo apt update  
$ sudo apt install python3-pip -y
```

2- Confirm pip3 is installed

```
$ pip3 --version
```

You should see something like:

```
$ pip 23.0.1 from /usr/lib/python3/dist-packages/pip (python 3.11)
```

3- Using of Python 3.12, need to install the matching venv package

```
$ sudo apt install python3.12-venv -y
```

Once **installed**, run the following to create and activate your virtual environment:

```
# Create  
$ python3.12 -m venv myenv  
# Activate  
$ source myenv/bin/activate  
# Install pandas  
$ pip install pandas
```

4- Create and Edit Files: Objective: Use text editors: nano for our scripts:

nano is a simple, user-friendly text editor that you use directly in the Linux terminal.

You can:

- ✚ Write and edit **Python** scripts (.py)
- ✚ Modify **configuration** files.
- ✚ Write notes or **README** files.
- ✚ Save and exit without needing a mouse.



Note: 1- Before you start, you need to ensure the files 'peptides_input.csv' and 'peptides.xlsx.csv' exist in your working directory. Copy the file from your PC to the Ubuntu directory.

```
$ cp /mnt/c/Users/lenovo/peptides_input.csv /home/khadijaelamin/
```

Explanation: Your PC's path contains the file, and your Ubuntu directory is where you want the file.

Note: To find out your current directory, use the command 'pwd'.

2- Then you need to **activate** your environment to run the script.

```
$ source myenv/bin/activate
```

Once **activated**, your shell prompt will change to show the environment name.

```
(myenv) khadijaelamin@DESKTOP-E1GF8D7:~$
```





1- Nano Editor: Peptide Allele Aggregator script:

```
$ nano group_peptides.py
```

✓ Write: “your script” inside the nano editor.

```
import pandas as pd

# Load the CSV file
df = pd.read_csv("peptides.xlsx.csv") # Replace with your actual file name

# Group by peptide and aggregate alleles into lists
grouped = df.groupby("peptide")["allele"].apply(list).reset_index()

# Remove duplicates, sort, and convert list to comma-separated string
grouped["alleles"] = grouped["allele"].apply(lambda alleles: ", ".join(sorted(set(alleles))))

# Count the number of unique alleles
grouped["allele_count"] = grouped["allele"].apply(lambda alleles: len(set(alleles)))

# Keep only desired columns: peptide, alleles, allele_count
final_df = grouped[["peptide", "alleles", "allele_count"]]

# Save to new CSV
final_df.to_csv("peptide_allele_grouped.csv", index=False, header=False)
```

- ✓ Save it by press: CTRL + O, Enter.
- ✓ Exit by: CTRL + X.
- ✓ **You can now execute the script using the following command:**

```
$ python group_peptides.py
```



Step-by-Step Explanation of the script:

✓ `import pandas as pd`

Imports the pandas (pd) library, which is essential for handling tabular data (like CSV or Excel files).

✓ `df = pd.read_csv("peptides.xlsx.csv")`

- Loads the CSV file into a DataFrame (df), a table-like structure used in pandas.
- df now contains your original data with at least two columns: "peptide" and "allele"

✓ `grouped = df.groupby("peptide")["allele"].apply(list).reset_index()`

- **Core step:** This **groups all rows** by the **peptide** value.
- For each unique peptide, it collects all corresponding "allele" values into a **list**.
- `.reset_index()` **converts** the result back into a normal **DataFrame** (instead of a grouped object).

✓ `grouped["alleles"] = grouped["allele"].apply(lambda alleles: ", ".join(sorted(set(alleles))))`

Cleans the allele list:

- `set(alleles)`: removes duplicates.
- `sorted(...)`: sorts the unique alleles alphabetically.
- `", ".join(...)`: joins them into a single **comma-separated string**.
- Saves this cleaned string into a new column called "alleles".

✓ `final_df = grouped[["peptide", "alleles", "allele_count"]]`

Counts how many unique alleles are associated with each peptide.

- The `len(set(...))` counts how many different alleles exist after duplicates are removed.
- Adds this number in a new column: "allele_count".



✓ `final_df = grouped[["peptide", "alleles", "allele_count"]]`

Extracts only the necessary columns:

- "peptide" – the sequence itself
- "alleles" – the cleaned, comma-separated list
- "allele_count" – the count of unique alleles

✓ `final_df.to_csv("peptide_allele_grouped.csv", index=False, header=False)`

- Exports the `final_df` table to a new CSV file.
- `index=False`: don't include pandas row numbers.
- `header=False`: don't include column names in the output CSV.

★ Core Parts of the Script

```
grouped = df.groupby("peptide")["allele"].apply(list).reset_index()
grouped["alleles"] = grouped["allele"].apply(lambda alleles: ", ".join(sorted(set(alleles))))
grouped["allele_count"] = grouped["allele"].apply(lambda alleles: len(set(alleles)))
```

These lines are the core of the script, as they carry out the transformation:

- ✚ Group alleles by peptide,
- ✚ Clean and format the allele lists,
- ✚ Count unique occurrences.

Example Input:

peptide	allele
HDSDSDTAKA	HLA-A*02:01
HDSDSDTAKA	HLA-B*07:02
HDSDSDTAKA	HLA-A*02:01

★ Example Output (CSV):

peptide	allele
AFHQATYNK	HLA-A*30:01
AGDPYYISK	HLA-A*11:01
AIYEVLOVA	HLA-A*02:01., HLA-A*02:03., HLA-A*02:06
APAEGEGAA	HLA-B*07:02., HLA-B*35:01
AQDSGASEL	HLA-A*02:06., HLA-B*40:01
ASSRAGKLF	HLA-B*57:01., HLA-B*58:01
ATYNKQPMY	HLA-A*01:01., HLA-A*03:01., HLA-A*11:01., HLA-A*26:01., HLA-A*30:01., HLA-A*30:02
AVISKSPSM	HLA-A*30:01., HLA-B*15:01
AYAPPPPAK	HLA-A*30:01
AYQYDDGYY	HLA-A*30:02
CEDSITVGM	HLA-B*40:01
CFLECAQYQ	HLA-A*30:02

Output

Row file



2- Nano Editor: Conical Peptide Generator script: Truncate peptides to a fixed length of 4.

```
$ nano generate_conical_peptides.py
```

```
import pandas as pd

# Load the CSV file
input_file = 'peptides_input.csv'      # Replace with your actual file name
df = pd.read_csv(input_file)

# Normalize column names
df.columns = df.columns.str.strip().str.lower()

# List to store new peptide records
conical_peptides = []

# Process each peptide
for index, row in df.iterrows():
    start = row['start']
    peptide = str(row['peptide']) # Ensure it's a string

    # Generate truncated peptides down to length 4
    for i in range(len(peptide), 3, -1): # from full length down to 4
        new_peptide = peptide[:i]
        new_end = start + len(new_peptide) - 1
        conical_peptides.append({
            'start': start,
            'end': new_end,
            'peptide': new_peptide,
            'number': len(new_peptide)
        })

# Convert to DataFrame and save
conical_df = pd.DataFrame(conical_peptides)
output_file = 'conical_peptides_output.csv'
conical_df.to_csv(output_file, index=False)

print(f"✅ Truncated peptides down to length 4 saved in: {output_file}")
```



- ✓ Save it by press: CTRL + O, Enter.
- ✓ Exit by: CTRL + X.
- ✓ **You can now execute the script using the following command:**

```
$ python generate_conical_peptides.py
```

Step-by-Step Explanation of the script:

- ✓ `import pandas as pd`

Imports the pandas (pd) library, which is essential for handling tabular data (like CSV or Excel files).

- ✓ `input_file = 'peptides_input.csv'`

your input file.

- ✓ `df = pd.read_csv(input_file)`

Read the input CSV file: this loads the data (containing peptide sequences with their start and end positions) into a DataFrame called (df)

- ✓ `df.columns = df.columns.str.strip().str.lower()`

Normalize column names by removing extra spaces and converting them to lowercase to prevent key errors (e.g., 'Start' vs 'start').

- ✓ `conical_peptides = []`

Initialize an empty list: this list will hold all the truncated peptide versions generated in the next step.

- ✓ `for index, row in df.iterrows():`

```
start = row['start']
```

- ✓ `peptide = str(row['peptide']) # Ensure it's a string`

Iterate through each row in the DataFrame: for each peptide:

Get its `start` position.

Convert the `peptide` to a string (in case it's read as a number or NaN).



✓ `for i in range(len(peptide), 3, -1): # from full length down to 4`

`new_peptide = peptide[:i]`

`new_end = start + len(new_peptide) - 1`

Create shortened versions of each peptide, starting from the full length down to a minimum of 4 characters. The new peptide uses the first `i` characters, and `new_end` is recalculated based on `start + (new peptide length - 1)`.

✓ `conical_peptides.append({`

`'start': start,`

`'end': new_end,`

`'peptide': new_peptide,`

`'number': len(new_peptide)`

`})`

Store each truncated version as a dictionary with:

- start: fixed start position
 - end: updated end based on truncated length
 - peptide: the truncated peptide sequence
 - number: length of the new peptide
- Each record is added to the `conical_peptides` list.

✓ `conical_df = pd.DataFrame(conical_peptides)`

`output_file = 'conical_peptides_output.csv'`

`conical_df.to_csv(output_file, index=False)`

The list of truncated peptides is converted into a new DataFrame called `conical_df`, which is then written to a file named `'conical_peptides_output.csv'`.

✓ `print(f" Truncated peptides down to length 4 saved in: {output_file}"):`

- ✓ Displays a confirmation message to inform the user that the task was successful and shows the output filename.



You should see something like:

```
(myenv) khadijaelamin@DESKTOP-E1GF8D7:~$ python generate_conical_peptides.py
Truncated peptides down to length 4 saved in: conical_peptides_output.csv
(myenv) khadijaelamin@DESKTOP-E1GF8D7:~$ ls
CPB_Program  conical_peptides_output.csv
(myenv) khadijaelamin@DESKTOP-E1GF8D7:~$
```

confirmation message

Example input

A	B	C	D
Start	End	Peptide	number
1	158	PRGKEERQEPSTTARKVGRPGRKRKHPPVESGDTPKDPAVISKSPSMAQDSGASELLPNGDLEKRSEPPQEEGSPAGGQKGGAPAEEGAAETLP	158
163	170	MEGSRGRL	8
179	183	SLRQR	5
190	198	FQAGDPYYI	9
200	203	KRRR	4
212	217	REAEEK	6
226	290	AVEENQGPGESQKVEEASPPAVQQPTDPASPTVATTPEPVGSDAGDKNATKAGDDEPEYEDGRGF	65
318	324	RSRAAEG	7
358	364	TYNKQPM	7
388	402	HDSDSDTAKAVEVQ	15
413	435	GFQPSGPKGLEPPEEKNPYKEV	23
440	468	WVEPEAAAYAPPPAKPRKSTAEKPKVK	29
527	534	QYDDDGQ	8
569	580	PGAAQAAIKEDP	12
599	602	EDWP	4
613	632	HDQEFDPKVPYPPVPAEKRK	20
708	711	SPCN	4
720	725	RKGLYE	6
741	749	ARPKFGDDR	9

Row file

★ Example Output (CSV):

A	B	C	D	E	F
293	440	451	WVEPEAAAYAPP	12	
294	440	450	WVEPEAAAYAP	11	
295	440	449	WVEPEAAAYA	10	
296	440	448	WVEPEAAAY	9	
297	440	447	WVEPEAAA	8	
298	440	446	WVEPEAA	7	
299	440	445	WVEPEA	6	
300	440	444	WVEPE	5	
301	440	443	WVEP	4	
302	527	534	QYDDDGQ	8	
303	527	533	QYDDDG	7	
304	527	532	QYDDDG	6	
305	527	531	QYDDD	5	
306	527	530	QYDD	4	
307	569	580	PGAAQAAIKEDP	12	
308	569	579	PGAAQAAIKED	11	
309	569	578	PGAAQAAIKF	10	

Output

Great, you can now copy the file to your PC and explore the results!

```
$ cp /home/khadijaelamin/ peptide_allele_grouped.csv /mnt/c/Users/lenovo/Desktop/
```

```
$ cp /home/khadijaelamin/ conical_peptides_output.csv /mnt/c/Users/lenovo/Desktop/
```



Alternatively, both Python scripts can be executed directly in Google Colab, which provides a ready-to-use Python environment by default, eliminating the need for a local Linux setup.

You just need to.

- ✓ Upload your data file in Google Colab.
- ✓ Install any required dependencies. Google Colab typically includes pandas by default; however, if it is not available in your environment, you can install it using the following command:

```
!pip install pandas
```

- ✓ Run the scripts.

✨ Good luck with your practice! ✨

Remember: Every expert was once a beginner. Each command you master brings you a step closer to success!